

DEPARTMENT OF UNIVERSITY
INSTITUTE OF COMPUTING
CHANDIGARH UNIVERSITY



SUBJECT NAME: DATA STRUCTURES USING C
CASE STUDY ON: CONTACT MANAGEMENT SYSTEM
SUBJECT CODE: 25CAT-151

SUBMITTED BY:

NAME: Somya Gupta

UID: 25BCD10029

SUBMITTED TO:

TEACHER NAME: Konika Rani Ma'am

DESIGNATION: AP

DATE OF SYBMISSION: 21/04/2026

▪ Index

Sr. No	Section	Page No
01	Title Page	01
02	Index	02
03	Abstract	03
04	Introduction	03
05	Objectives of the Study	03
06	Fundamentals of Contact Management System	04
07	System Architecture & Data Flow	05
08	Module-Wise Explanation of Source Code	06
09	Algorithmic Complexity Analysis	07
10	Sample Input and Expected Output	07 – 08
11	Limitations of the Project	08
12	Future Trends and Innovations	09
13	Learning Outcomes	09 – 10
14	Conclusion	10
15	References	10

1. Abstract

This case study examines the design and implementation of a **Contact Management System** developed in the C programming language as part of the Data Structures Lab (25CAP-152) at Chandigarh University. The project employs a singly linked list as its core data structure for dynamic memory management, and implements three key algorithms: insertion sort, linear search, and binary search to deliver an efficient, menu-driven console application.

The system supports six operations: adding contacts with duplicate prevention, displaying all contacts, deleting contacts with proper memory deallocation, alphabetical sorting, fast binary search, and graceful program exit. This study analyzes the architecture, module-level logic, algorithmic complexity, and practical outcomes of the project, while identifying limitations and proposing meaningful future enhancements grounded in modern software practices.

2. Introduction

Contact management is a fundamental requirement in both personal and professional environments. Before the advent of digital systems, physical diaries and address books were the primary storage mediums, methods prone to data loss, difficult to update, and entirely inefficient for searching large volumes of information. With the rapid growth of computing from the 1980s onward, digital contact managers emerged as structured, scalable alternatives.

This project implements a Contact Management System in C, a language well-suited for systems-level programming due to its direct memory control, pointer arithmetic, and minimal runtime overhead. The application is a console-based, menu-driven program that allows users to add, display, delete, sort, and search contacts in real time. Rather than relying on a static array, contacts are stored in a dynamically growing singly linked list, which means memory is only allocated when a new contact is added and freed as soon as one is removed.

What makes this project particularly educational is that it is built entirely from first principles. There are no external libraries for sorting, searching, or data management; every operation is hand-coded. This makes it an ideal academic project for demonstrating how data structures and algorithms solve real-world problems, and how efficient data handling directly impacts system performance. It also introduces the programmer to concepts like pointer manipulation, heap memory management, and algorithm design in a tangible, applied context that goes beyond textbook theory.

3. Objectives of the Study

- Design and implement a functional Contact Management System entirely in C using the singly linked list data structure.
- Understand and apply dynamic memory allocation using `malloc()` and `free()` for runtime contact storage and removal.
- Implement linear search for duplicate prevention to maintain data integrity across all insertions.
- Apply insertion sort directly on linked list node pointers for in-place alphabetical ordering without auxiliary arrays.
- Implement binary search on a pointer array derived from the linked list, achieving $O(\log n)$ lookup efficiency.
- Build a clean, user-friendly, menu-driven console interface that handles invalid inputs and edge cases gracefully.
- Analyze the time and space complexity of each operation and understand the trade-offs between different data structures and algorithms.
- Develop practical debugging, logical thinking, and modular C programming skills applicable to larger software systems.

4. Fundamentals of Contact Management System

The system is architecturally simple but algorithmically rich. It is built on three pillars: the data structure that stores contacts, the algorithms that manipulate them, and the memory management strategy that keeps the program efficient and safe.

4.1 Data Structure — Singly Linked List

Each contact is represented as a node in a singly linked list. The C structure defining a node contains three character array fields - name (50 chars), phone (15 chars), and email (50 chars) — along with a self-referential pointer (struct Contact* next) that links it to the next node in the chain. A single global pointer, head, points to the first node in the list.

This design offers several advantages over a static array. First, there is no predefined size limit — the list can hold as many contacts as the system's available heap memory allows. Second, insertion at the head is $O(1)$, since it only requires updating the head pointer. Third, deletion is efficient once the target node is found, as it only requires adjusting one pointer and freeing one memory block. The trade-off is that random access is not possible — reaching the n th element requires traversing all preceding nodes.

4.2 Algorithms Used

Linear Search (isDuplicate): Before every insertion, the system traverses the full linked list using strcmp() to compare the new contact's name against every existing node. If a match is found, the function returns 1 and the insertion is aborted, preventing duplicate entries. This runs in $O(n)$ time in the worst case.

Insertion Sort on Linked List (sortContacts): The sorting algorithm processes each node of the original list one by one, removing it and reinserting it into its correct alphabetical position in a separate sorted list. This avoids the need to copy data into an array and works entirely by rearranging node pointers. While the worst-case time complexity is $O(n^2)$, it is well-suited for the expected small-to-medium dataset sizes of a contact list.

Binary Search (binarySearchContact): Because binary search requires $O(1)$ random access by index, it cannot operate directly on a linked list. A helper function (listToArray) first copies all node pointers into a temporary array. Binary search then applies the standard divide-and-conquer approach on this array, halving the search space at each step and achieving $O(\log n)$ time complexity. This combination of listToArray + binarySearch delivers fast lookup when the list is sorted.

4.3 Memory Management

Every contact node is heap-allocated using malloc(sizeof(struct Contact)). When a contact is deleted, free(temp) releases the memory back to the operating system. This explicit memory management is one of C's most powerful features — and one of its most error-prone. This project handles it correctly by always pairing every malloc() with a corresponding free(), and by freeing the pre-allocated node in addContact() if a duplicate is detected before the node is ever inserted into the list.

4.4 User Interaction Model

The program runs in an infinite while(1) loop in main(), presenting a numbered menu at each iteration. User input is read via scanf(), and a switch statement routes each choice to the corresponding function. The loop only terminates when the user selects option 6, which calls exit(0) to cleanly terminate the process. This event-driven, menu-loop pattern is a standard design for console applications in C and forms the backbone of many embedded system interfaces.

5. System Architecture & Data Flow

Understanding how data flows through the system from user input to memory and back is essential for appreciating the design decisions made in this project. The following describes the full lifecycle of a contact within the system.

5.1 Contact Lifecycle

Creation: When the user selects option 1, addContact() calls malloc() to allocate a new node on the heap. User input (name, phone, email) is read into the node's fields. isDuplicate() is immediately called if a duplicate is found, the allocated memory is freed and the function returns without insertion. If no duplicate exists, the new node is prepended to the list by setting newNode->next = head and then head = newNode.

Storage: The contact lives in heap memory, connected to the rest of the list via its next pointer. The global head pointer serves as the entry point for all list operations. Every function that needs to traverse or modify the list starts from head and follows the chain of next pointers.

Sorting: When the user sorts, sortContacts() rebuilds the entire list in alphabetical order by extracting each node and reinserting it. After sorting, head points to the alphabetically first contact, and the chain follows in order. This is a destructive operation on the original list order it cannot be undone without re-adding contacts.

Searching: After sorting, binarySearchContact() copies node pointers (not data) into a stack-allocated array of 100 elements. Binary search is applied on this array. The array is temporary and exists only for the duration of the function call it does not affect the linked list itself.

Deletion: deleteContact() traverses the list with two pointers (temp and prev). When the target node is found, prev->next is updated to skip over temp, effectively removing it from the chain. free(temp) then releases the heap memory, completing a clean deletion with no memory leak.

5.2 Program Flow Diagram (Textual)

START -> Display Menu -> Read Choice

Choice 1 (Add): malloc() -> Read Input -> isDuplicate() -> [Yes: free() + return] / [No: Insert at head]

Choice 2 (Display): Traverse list from head -> Print each node -> Return to menu

Choice 3 (Delete): Traverse with prev/temp -> Find target -> Relink pointers -> free(temp)

Choice 4 (Sort): Extract each node -> Reinsert in sorted position -> head = sorted list

Choice 5 (Search): listToArray() -> binarySearch() on array -> Print result or 'Not Found'

Choice 6 (Exit): exit(0) -> Program terminates

6. Module-Wise Explanation of Source Code

Module / Function	Algorithm / Logic	Time Complexity	Purpose
isDuplicate()	Linear Search	O(n)	Traverses the list and uses strcmp() to detect an existing contact with the same name before insertion.
addContact()	O(1) Head Insertion	O(n) overall (due to duplicate check)	Allocates a new node via malloc(), reads user input, checks for duplicates, and prepends to the list.
displayContacts()	List Traversal	O(n)	Iterates from head to NULL and prints name, phone, and email of every contact. Handles empty list gracefully.
deleteContact()	Two-Pointer Search + Relinking	O(n)	Locates the target node, adjusts the previous node's next pointer to bypass it, then calls free() to release heap memory.
sortContacts()	Insertion Sort on Linked List	O(n ²) worst case	Removes each node from the original list and reinserts it into the correct alphabetical position in a new sorted list.
listToArray()	List Traversal to Array	O(n)	Copies all node pointers into a temporary array to enable index-based access required by binary search.
binarySearchContact()	Binary Search on Pointer Array	O(log n)	Performs divide-and-conquer search on the sorted pointer array. Prints full contact details if found.
main()	Infinite Loop + Switch Dispatch	O(1) per iteration	Runs a menu loop and routes each user selection to the appropriate function via a switch statement.

7. Algorithmic Complexity Analysis

A well-designed system not only works correctly but also performs efficiently. The following analysis evaluates each operation in terms of time and space complexity, which is critical for understanding the scalability of the system.

Operation	Best Case	Average Case	Worst Case	Space Complexity
Add Contact	$O(1)^*$	$O(n)$	$O(n)$	$O(1)$ extra
Display All	$O(1)$ if empty	$O(n)$	$O(n)$	$O(1)$ extra
Delete Contact	$O(1)$ if head	$O(n)$	$O(n)$	$O(1)$ extra
Sort (Insertion Sort)	$O(n)$ if sorted	$O(n^2)$	$O(n^2)$	$O(1)$ extra
Binary Search	$O(1)$ if mid	$O(\log n)$	$O(\log n)$	$O(n)$ for array
Duplicate Check	$O(1)$ if first	$O(n)$	$O(n)$	$O(1)$ extra

* Best case for Add is $O(1)$ only if the duplicate check is skipped. In practice, every add performs a full $O(n)$ linear scan for duplicates.

The $O(n)$ space used by binary search refers to the temporary pointer array created in `listToArray()`. This array is stack-allocated and freed automatically when the function returns, so it has no long-term memory impact. However, the hardcoded size of 100 elements does represent a fixed upper bound that could be exceeded in a production environment.

It is also worth noting that sorting must be performed before binary search is used. If the user attempts binary search on an unsorted list, the results will be incorrect. This is a known design constraint of the current implementation it places the responsibility on the user to sort before searching, rather than handling this automatically in code.

8. Sample Input and Expected Output

The following walkthrough demonstrates a complete, realistic user session with the system, covering all six menu operations. Each step shows the exact user input and the corresponding system output.

8.1 Session Walkthrough

Step	User Action	System Output
1	Select Option 1. Enter Name: Somya, Phone: 7973704290, Email: somya2412@gmail.com	Contact Added Successfully!
2	Select Option 1. Enter Name: Mrithul, Phone: 8765392100, Email: mrithul2004@gmail.com	Contact Added Successfully!
3	Select Option 1. Enter Name: Somya (duplicate attempt)	Contact already exists! (node freed, not inserted)
4	Select Option 2 (Display All)	Mrithul 8765392100 mrithul2004@gmail.com Somya 7973704290 somya2412@gmail.com

5	Select Option 4 (Sort)	Contacts Sorted Successfully!
6	Select Option 2 (Display after sort)	Mrithul 8765392100 mrithul2004@gmail.com Somya 7973704290 somya2412@gmail.com
7	Select Option 5 (Binary Search). Enter name: Mrithul	Found: Mrithul 8765392100 mrithul2004@gmail.com
8	Select Option 3 (Delete). Enter name: Somya	Contact Deleted Successfully!
9	Select Option 6 (Exit)	=== Code Execution Successful ===

8.2 Edge Case Behaviour

The system handles several important edge cases that distinguish a robust implementation from a naive one:

- **Empty List Display:** If no contacts exist and the user selects Display, the system prints "No contacts available!" and returns to the menu without crashing.
- **Delete Non-Existent Contact:** If the user attempts to delete a name that does not exist in the list, the system prints "Contact not found!" without modifying the list or causing undefined behaviour.
- **Binary Search on Empty List:** If the user attempts a binary search with zero contacts, `listToArray()` returns 0 and the system prints "No contacts available!" before exiting the function safely.
- **Duplicate Prevention Memory Safety:** When a duplicate is detected, the node pre-allocated by `malloc()` is explicitly freed before returning preventing a memory leak that would occur if the allocated pointer were simply abandoned.

9. Limitations of the Project

While the project successfully meets its academic objectives, several practical limitations exist that would need to be addressed in a production-grade system:

- **No Persistent Storage:** All contact data resides in heap memory and is lost when the program exits. There is no file I/O (`fread`, `fwrite`, `fprintf`) or database integration, making the system impractical for real-world use where data must survive across sessions.
- **Fixed Binary Search Array:** The `listToArray()` function uses a hardcoded array of 100 pointer elements. Any contact list exceeding 100 entries would cause a buffer overflow, a critical memory safety issue in C.
- **No Input Validation:** Phone numbers and email addresses are stored as raw strings with no format checking. A user could enter "abc" as a phone number or a string without "@" as an email, and the system would accept it without complaint.
- **Case-Sensitive Operations:** The `strcmp()` function used throughout the system is case-sensitive. This means "Somya" and "somya" are treated as entirely different contacts a significant usability flaw that would confuse real users.
- **Name-Only Search:** The system only supports contact lookup by name. Searching by phone number or email address common real-world requirements is not implemented.
- **Sort Before Search Dependency:** Binary search produces incorrect results if applied to an unsorted list. The system does not warn the user about this constraint or automatically sort before searching.
- **Console-Only Interface:** The lack of a graphical user interface limits accessibility for non-technical users. All interaction is via numbered menu options and terminal text output.

10. Future Trends and Innovations

The current implementation provides a solid foundation that can be extended in several meaningful directions, ranging from quick code-level improvements to architectural redesigns:

10.1 Short-Term Improvements (C-level)

- **File I/O Persistence:** Adding `fopen()`, `fwrite()`, and `fread()` to save contacts to a binary file on exit and reload them on startup would make the system genuinely useful across sessions.
- **Dynamic Binary Search Array:** Replacing the hardcoded `array[100]` in `listToArray()` with a `malloc'd` array sized to the actual contact count (using a counter variable) would eliminate the 100-contact ceiling and buffer overflow risk.
- **Case-Insensitive Comparison:** Replacing `strcmp()` with `strcasecmp()` (on Linux/macOS) or a custom `toLower()` conversion before comparison would make searches and duplicate checks case-insensitive.
- **Input Validation:** Adding basic checks - `isdigit()` for phone fields, `strchr(email, '@')` for email fields would prevent malformed data from entering the system.

10.2 Architectural Upgrades

- **Hash Table for $O(1)$ Search:** Replacing the linked list with a hash table using contact name as the key would reduce average search time from $O(n)$ to $O(1)$, dramatically improving performance for large datasets.
- **Doubly Linked List for Bidirectional Traversal:** Adding a `prev` pointer to each node would enable reverse traversal and make deletion $O(1)$ if a direct pointer to the node is available.
- **SQLite Database Integration:** Embedding SQLite into the C project would provide structured queries, persistent storage, and unlimited contacts without reimplementing data management logic.

10.3 Modern Application Development

- **GUI Frontend:** Porting the core logic to Python with a Tkinter or PyQt interface, or to JavaScript with a React-based web UI, would make the system accessible to non-technical users.
- **REST API:** Exposing the contact operations as HTTP endpoints using a lightweight C framework or rewriting in Node.js/Flask would enable multi-user access and mobile app integration.
- **Cloud Sync:** Integrating with a cloud database like Firebase or MongoDB Atlas would allow contacts to be accessed and updated from multiple devices in real time the standard expectation of modern contact applications.

11. Learning Outcomes

The development of this project provided hands-on exposure to a wide range of computer science concepts that are difficult to fully appreciate through theory alone. The following outcomes were achieved:

1. **Singly Linked List Mastery:** Gained deep understanding of node creation, dynamic list growth, pointer-based traversal, head insertion, and the two-pointer deletion technique. These are skills directly transferable to implementing other pointer-based structures such as stacks, queues, and trees.
2. **Dynamic Memory Management in C:** Developed practical competency with `malloc()` and `free()`, including the discipline of always pairing allocations with deallocations and handling the case where memory must be freed before a node is inserted (duplicate rejection scenario).
3. **Algorithm Implementation from Scratch:** Implemented three distinct algorithms - linear search, insertion sort (on a linked list, not an array), and binary search - without using any library functions, building genuine algorithmic intuition.

4. Complexity Trade-off Analysis: Understood concretely why binary search requires an array (indexed access), why insertion sort is $O(n^2)$ but acceptable for small inputs, and how the choice of data structure directly constrains which algorithms are usable.
5. Modular C Programming: Designed a program where each function has a single, well-defined responsibility. This separation of concerns makes code easier to test, debug, and extend a principle central to software engineering at all levels.
6. Edge Case Handling and Defensive Programming: Learned to anticipate and gracefully handle failure scenarios: empty lists, missing contacts, duplicate insertions, and pre-search memory cleanup. These practices are what distinguish reliable software from fragile code.
7. Debugging Pointer-Based Code: Developed practical skills in tracing pointer chains, identifying dangling pointer risks, and using logical reasoning to verify that list modifications produce the correct structural outcome.

12. Conclusion

The Contact Management System developed in C is a compact but algorithmically rich project that demonstrates the direct applicability of fundamental data structures and algorithms to real-world software problems. Through the use of a singly linked list, the system achieves dynamic, memory-efficient storage that adapts to the number of contacts at runtime. The combination of three algorithms linear search for integrity, insertion sort for ordering, and binary search for fast retrieval makes the system both functionally complete and educationally comprehensive.

The project goes beyond a simple coding exercise. It forces the developer to think about memory lifecycle (when to allocate, when to free), algorithmic constraints (why binary search needs an array), and design trade-offs (head insertion for speed vs. sorted insertion for order). These are exactly the kinds of decisions that software engineers face at every level of the industry, from embedded firmware to large-scale distributed systems.

The limitations identified lack of persistence, case-sensitivity, fixed array size are not flaws in the academic context but deliberate scope boundaries that highlight exactly where the project could evolve. Each limitation maps to a real software engineering concept: file I/O, string normalization, dynamic resizing. In this way, the project serves not only as a working system but as a roadmap for further learning.

In summary, this case study affirms that the principles of data structures and algorithms are not abstract academic concepts they are the building blocks of every piece of software that manages, organizes, and retrieves information. This project makes those principles concrete, visible, and deeply understood.

13. References

- Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Kanetkar, Y. (2016). Let Us C (15th ed.). BPB Publications.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley. — Reference for insertion sort and binary search complexity analysis.
- GeeksforGeeks. (2024). Linked List Data Structure. <https://www.geeksforgeeks.org/linked-list-data-structure/>
- GeeksforGeeks. (2024). Insertion Sort for Linked List. <https://www.geeksforgeeks.org/insertion-sort-for-singly-linked-list/>
- cppreference.com. (2024). C Standard Library — string.h, stdlib.h, stdio.h. <https://en.cppreference.com/w/c>
- GitHub Repository: Somya Gupta. (2026). Contact Management System Using C. <https://github.com/Somya-ds/Contact-Management-System-Using-C>